

**YTPM**

YOUTUBE PLAYLIST MANAGER

ISHA ANALYTIKS LLC · DESKTOP PRODUCT GUIDE

Command your YouTube playlists with precision — and spend quota only when something actually changes.

YouTube Playlist Manager (YTPM) is a desktop workspace for owners who maintain large, private, and unlisted playlists. Connect a Google account once, then create, clean, sort, move, and archive lists without fighting YouTube Studio's limits. On Windows you can unzip a packaged build — no Python on the user PC.

OAuth desktop app

YouTube Data API v3

Sparse writes

Dry-run

Local cache

10,000 unit daily pool

Windows zip

Product snapshot

Product	YouTube Playlist Manager (YTPM)
Publisher	Isha Analytiks LLC
Audience	Channel owners managing their own playlists
Auth	Google OAuth · scope youtube.force-ssl
Quota	Reads $\approx$ 1 unit · writes = 50 units each

Purpose

YTPM exists because YouTube Studio is slow for bulk playlist work: alphabetical order, duplicate titles after extra spaces, broken rows, renaming videos you uploaded, and moving videos across many lists. The app talks to **your** channel through the official YouTube Data API. It never needs a second Google Cloud project to “multiply” quota, and it never empties a playlist as a sort strategy.

Every expensive action is designed to be **sparse**: update a title only if consecutive spaces are still there (or you explicitly rename), delete a row only if it is a duplicate, and rewrite a position only if the item is not already in the right slot. An already-clean, already-sorted playlist costs **zero write quota**. Sort, Clear, and List Mode Apply can **dry-run** first (0 write units). The sidebar quota meter is a local ledger — Google does not expose remaining Data API quota.

Install (Windows)

Use a Python venv from this repo, or unzip a packaged folder. Client ID and Secret live in `.env`. After Connect Google Account, `token.json` is written next to the app — do not share it.

From source

Set `YTPM_VENV` in `.env`. Prefer `Run_YTPM.vbs` (no CMD window). Missing packages install on startup. CLI: `Run_YTPM_CLI.bat`.

Packaged zip

Keep `YTPM.exe` next to `_internal`. Put Client ID/Secret in `.env` beside the exe (the exe ignores `YTPM_VENV`). Double-click, then Connect Google Account. Each person signs in with their own Google account.

Layout on first launch

Playlists and Videos each get **50%** of the main pane. Drag the split; closing the app saves window size, split, filters, and last playlist for next time.

Features

Account gate

Connect Google Account on first launch. Token stays local. Reconnect when you switch accounts. TLS uses the OS certificate store on Windows. A second launch focuses the existing window. Source launchers read `YTPM_VENV` from `.env` and install missing packages. Packaged `YTPM.exe` ignores the `venv` — put `.env` next to the exe.

Playlist command center

Create, rename, description, privacy, delete, and clear lists. Filter as you type (remembered). Right-click rows. Delete requires a checkbox that the wipe cannot be undone. `F2` / `Del` / `Ctrl+F` / `Enter` shortcuts.

Intelligent sort

Sort by Title, Date Added, Duration, or Channel. The sidebar queue shows playlist, key, and pending/done — drop or reorder jobs, then resume after Pacific midnight.

Hygiene

Trim duplicate video ids. Find rows YouTube labeled deleted or inaccessible. Remove selected videos from this playlist without deleting them on YouTube.

List Mode

Export selected playlists to a text file, edit offline, apply the diff when quota allows (or dry-run with 0 writes). Undo picks among the last 20 snapshots.

Library tools

Rename titles on videos you uploaded. Searchable copy/move destination. Each write is logged (id, method, units) next to the local quota meter. First launch: playlists/videos 50/50; your split, window size, and filters are saved on exit.

Button reference

Costs assume **one playlist of 50 videos**, YouTube Data API v3, and the default **10,000 units/day** pool (resets midnight US Pacific). The sidebar quota meter is a **local estimate** (Google does not expose remaining Data API quota).

Method	Units
<code>playlists.list</code> , <code>playlistItems.list</code> , <code>videos.list</code> , <code>channels.list</code>	1 per request (50 results/page)
<code>playlists.insert / update / delete</code>	50
<code>playlistItems.insert / update / delete</code>	50
<code>videos.update</code>	50

Reads are cheap. Writes are **50 units each**. OAuth sign-in is **not** billed against this quota.

After several actions the app also **Refreshes** (listed separately below). Those extra units are included in the Cost column when the button always refreshes.

Dry run (Sort / Clear / List Mode Apply): Yes = apply, No = dry run (writes the list-mode file, **0 write quota**), Cancel = abort.

Rename Playlist, Rename title, and Delete Playlist open with the **current name already in the box**. Ok with the same playlist/video title skips the write.

Button	Purpose	Functionality	YouTube API Costs (50-video playlist)
Connect Google Account (sidebar and startup banner)	Sign in so the app can read and edit your playlists.	Opens the Google OAuth browser flow, stores a local token, then loads playlists. Same action as the banner button.	0 Data API units for OAuth. After success it runs Refresh (~see Refresh).
Reconnect / Switch Account	Change Google account or force a new consent screen.	Clears the cached token, runs OAuth again, then loads playlists.	0 Data API units for OAuth. After success it runs Refresh.
Quota meter (sidebar label)	Show ~units left today.	Reads local <code>quota_ledger.json</code> (Pacific date). Not a Google API field. Each write is also logged as <code>WRITE method id=... units=50</code> so you can match Google Cloud's quota graph.	0 units .
Sort queue (sidebar table)	See pending/done sort jobs; drop or reorder.	Playlist title, sort key, status. Drop removes a job. Up/Down change run order. Resume unfinished sorts still runs pending jobs in this order.	0 units (local).
Refresh	Reload the playlist list from YouTube and drop the local video cache.	Calls <code>playlists.list</code> (paginated). Selecting a playlist then loads its videos (<code>playlistItems.list</code> + <code>videos.list</code>) unless a cache hit.	Playlists: 1 unit per 50 playlists you own. Loading the 50-video playlist: ~2 units . Typical

Button	Purpose	Functionality	YouTube API Costs (50-video playlist)
			total ~ 3 units . Cache hit when switching playlists: 0 .
Filter playlists... (table search)	Find year-named lists without scrolling.	Filters the playlists table by title or id as you type. Ctrl/Shift multi-select is kept for rows that stay visible.	0 units (local).
Filter videos... (table search)	Find a video in the current playlist.	Filters the videos table by title, channel, or video id as you type.	0 units (local).
New Playlist	Create an empty playlist.	Prompts for a title (empty box); <code>playlists.insert</code> (private); then Refresh.	50 to create + Refresh (~ 3). Total ~ 53 . Independent of video count (new list is empty).
Rename	Change the selected playlist's title.	Box is pre-filled. <code>playlists.list</code> then <code>playlists.update</code> ; then Refresh. Same title: no write. F2 on the playlists table.	1 + 50 + Refresh ~ 3 . Total ~ 54 . Unchanged name: 0 .
Description...	Edit the playlist description.	Pre-filled text box. <code>playlists.update</code> (keeps title and privacy). Unchanged text: list only.	1 if unchanged; 1 + 50 if saved.
Privacy	Set public / unlisted / private.	One <code>playlists.update</code> on the focused playlist.	50 .
Delete Playlist	Permanently delete the selected playlist.	Box is pre-filled. Ok requires the checkbox I understand this cannot be undone and a matching title. One <code>playlists.delete</code> (videos stay on YouTube); then Refresh.	50 + Refresh ~ 3 . Total ~ 53 . Does not cost 50×50 to remove items.
Clear Videos	Empty every selected playlist but keep the playlists.	Quota confirm + dry run. Snapshots for Undo, then <code>playlistItems.delete</code> on every row.	1 (list) + 50 × 50 deletes = 2,500 per playlist. Dry run: list + snapshot file, 0 writes .
Trim Duplicates	Keep the first copy of each video id; remove later repeats.	Lists items, deletes duplicate playlist rows only. Applies to every selected playlist.	Reads: ~ 1–2 + 1 (items). Writes: 50 × (number of duplicate

Button	Purpose	Functionality	YouTube API Costs (50-video playlist)
			rows). No duplicates: ~3.
Sort by Title	A–Z after collapsing extra spaces in titles you own and removing duplicates.	Queued sparse sort (see Notes). Dry run writes list-mode with 0 writes. Per-item progress. Studio Sort → Manual for in-place moves; otherwise a new sorted copy.	Best (already A–Z, clean): 0 writes + ~2–4 reads. Typical: 50 × (collapsed titles + deleted dupes + moved items). Copy fallback: ~2,550 plus any title/dupe writes.
Sort by Date Added	Oldest-first by date added, after the same cleanup as Sort by Title.	Same sparse writes. Duplicate rows keep the oldest copy. Copy fallback if Sort is not Manual. A later sort on the same playlist replaces the queued key (Title → Date).	Same write rules as Sort by Title.
Sort by Duration	Shortest-first (same 50-unit position writes).	Enriches durations (<code>videos.list</code>), then the same sparse move/dedupe path. Still needs Studio Sort → Manual .	Enrich ~1 extra read + same write rules as Sort by Title.
Sort by Channel	Channel name A–Z (same 50-unit position writes).	Enriches channel titles, then sparse moves. Still needs Studio Sort → Manual .	Enrich ~1 extra read + same write rules as Sort by Title.
Resume unfinished sorts	Continue the sort job queue after quota runs out.	List Mode already saved intended order. Stops when local remaining units < 50. Resume after Pacific midnight.	Same as the remaining sort writes. 0 if the queue is empty.
Undo...	Restore the focused playlist from a chosen list-mode snapshot.	Picker lists the last 20 snapshots (newest first). Applies that file (same writes as List Mode Apply).	Same as List Mode Apply for that playlist.
List Mode: Export selected	Save selected playlists as tab-separated text files.	<code>playlistItems.list</code> for each selected playlist only (not every playlist you own).	~1 per selected playlist (50 videos = 1 page). 0 writes.

Button	Purpose	Functionality	YouTube API Costs (50-video playlist)
List Mode: Apply	Push edits from those text files back to YouTube.	Quota confirm + dry run. Diffs each file, then deletes/inserts/moves.	Per 50-video file: ~2 lists . Writes: 50 × (deletes + inserts + moved items) . Dry run: reads only.
Export JSON	Snapshot every selected playlist to JSON.	Lists items, <code>videos.list</code> for metadata, writes under the export folder.	~2 units per selected playlist. 0 writes .
Find Broken	Find deleted / private-unavailable rows in every selected playlist.	Lists items, enriches, reports placeholder titles. Optionally removes those playlist rows.	Scan: ~2 units per playlist. Removing *B* rows: +50 × B .
Stats	Duration totals and unique channels for every selected playlist.	Lists items, <code>videos.list</code> for durations, combined dialog.	~2 units per selected playlist. 0 writes .
Tutorials / Help	Open this product guide in the browser.	Opens local <code>docs.html</code> (no YouTube API). Available even when not connected.	0 units .
Open in Browser	Watch the selected video on YouTube.	Opens <code>https://www.youtube.com/watch?v=...</code> for the first selected row. Double-click / o key.	0 units .
Open in Studio	Open the focused playlist in YouTube Studio.	<code>https://studio.youtube.com/playlist/{id}/videos</code> (set Sort → Manual here). Enter on a playlist or video row.	0 units .
Rename title...	Change the YouTube title of a selected video you uploaded.	Box is pre-filled. <code>videos.update</code> . Not a playlist-only nickname — the title changes everywhere. Other channels are rejected. Same title: no write. F2 on the videos table.	1 (<code>videos.list</code>) + 50 write. Already the same title: 0 writes .

Button	Purpose	Functionality	YouTube API Costs (50-video playlist)
Remove	Remove selected videos from this playlist.	Confirms, <code>playlistItems.delete</code> for each selected playlist item (videos stay on YouTube), reloads. Del on the videos table.	50 × (selected rows) + reload ~2.
Copy...	Copy selected videos onto another playlist (skips ids already there).	Searchable destination picker (filter as you type; IDs shown so duplicate titles cannot collide). Destination cache is dropped so the dest list is fresh next time you open it.	Destination list: 1 . Writes: 50 × (videos actually copied).
Move...	Move selected videos from this playlist to another.	Same searchable picker; insert into dest then delete from source; reloads source.	100 × (moved videos) + reload ~2.

Notes

- **Selecting a playlist** in the left table costs **~2 units** the first time (`playlistItems.list` + `videos.list`). Switching back uses the **local item cache (0 units)** until Refresh or a mutation.
- **Column headers** (Title, Items, Duration, ...) sort the table **locally** only: **0 units**. On startup, Refresh, and after a filter rebuild, Playlists default to **Title ▲** and Videos to **# ▲**. Click another heading to change it until the next rebuild.
- **Multi-select playlists** (Ctrl/Shift): Trim, Sort, Clear, Find Broken, Export JSON, Stats, and List Mode Export selected all run on **every selected playlist**.
- **Sort by Title / Date / Duration / Channel** share the same sparse writes:
- **Titles:** `videos.update` only if the title still has extra spaces **and** you own the video.
- **Duplicates:** `playlistItems.delete` only when a duplicate is found (same video id, or same title after space-collapse). Placeholder titles (Deleted video / Private video) are **not** merged with each other. Date sort keeps the **oldest** copy.
- **Order:** `playlistItems.update` only for items that are not already in the right slot. Already-sorted playlists cost **0 write quota**.
- Set Studio **Sort** → **Manual** (Open in Studio) before sorting so the app can patch positions in place. Otherwise it spends copy-fallback quota (~2,550 for 50 videos) and does not reorder the original.
- There is no cheaper official reorder method. **10,000 units/day ÷ 50 = 200 writes**. Do not use unofficial YouTube web APIs to dodge quota.
- **Keyboard:** **F2** rename (playlist or video title, last-clicked table), **Del** remove selected videos, **Ctrl+F** focus the videos filter, **Enter** open the playlist in Studio. Right-click a row for rename, privacy, copy/move, and Studio.
- Window size, the playlist/videos split, both filter boxes, and the last playlist are remembered in `gui_settings.json` (saved when you close the app). **First launch** (or a missing/invalid split): Playlists and Videos each get **50%** height. After you drag the divider, the ratio (`sash_frac`) is restored on the next start. A second `Run_YTPM.vbs` / `YTPM.exe` focuses the existing window instead of stacking another.
- Launch: Prefer `Run_YTPM.vbs` (no CMD window). It reads `YTPM_VENV` from `.env`, runs `ytpm_launch.pyw` under `pythonw`, and installs missing packages via `ensure_deps` (pip uses `CREATE_NO_WINDOW`). `Run_YTPM.bat` delegates to the `.vbs`. CLI/Build still use `ytpm_resolve_venv.bat`. Packaged **YTPM.exe** ignores `YTPM_VENV` — put `.env` next to the exe. Team zip: `Build_YTPM.bat --zip` (see `docs/distribute.md`).
- CLI (same ops as the GUI): `ytpm sort PLAYLIST --by title` (or date, duration, channel) with optional `--dry-run` / `--reverse`; `ytpm sort --resume`; `ytpm items rename VIDEO_ID "New title"`; `ytpm playlists description PLAYLIST "Text"`; `ytpm list export [ids...]`; `ytpm list apply [--dry-run]`; `ytpm playlists privacy PLAYLIST public`; `ytpm stats PLAYLIST`; `ytpm version`. See the CLI section (`docs/cli.md`).

CLI

Same YouTube Data API v3 operations as the GUI. Prefer `Run_YTPM_CLI.bat` (reads `YTPM_VENV` from `.env` and runs `ensure_deps.py` first). Or use `python -m ytpm` from that same venv. Global flag: `-v` / `--verbose` (debug logging).

Auth and GUI

Command	What it does
<code>ytpm auth</code>	Google OAuth browser consent. <code>--force</code> reconnects / switches account.
<code>ytpm auth --force</code>	Clears the token and re-consents.
<code>ytpm gui</code>	Launches <code>ytpm_gui.py</code> .
<code>ytpm version</code>	Prints the package version (also used in <code>release\YTPM-<version>-windows.zip</code>).

OAuth is **0 Data API units**. Writes after that follow the same 50-unit costs as the GUI.

Playlists

Command	What it does
<code>ytpm playlists</code>	List owned playlists.
<code>ytpm playlists create "Title"</code>	Create a private playlist (50).
<code>ytpm playlists rename PLAYLIST "New title"</code>	Rename (1 + 50).
<code>ytpm playlists privacy PLAYLIST public</code>	One <code>playlists.update</code> (50). Also unlisted or private.
<code>ytpm playlists description PLAYLIST "Text"</code>	Update description (1 if unchanged, else 1 + 50).
<code>ytpm playlists delete PLAYLIST</code>	Type-to-confirm delete, or <code>--yes --confirm-name "Title"</code> (50).
<code>ytpm playlists clear PLAYLIST</code>	Remove all items. <code>--dry-run</code> snapshots only (0 writes). <code>--yes</code> skips the prompt.

PLAYLIST may be a playlist id or an exact title.

Items

Command	What it does
<code>ytpm items add PLAYLIST VIDEO_ID</code>	Insert one video (50).
<code>ytpm items remove PLAYLIST_ITEM_ID</code>	Delete that playlist row (50). Video stays on YouTube.
<code>ytpm items rename VIDEO_ID "New title"</code>	<code>videos.update</code> on a video you uploaded (1 + 50). Changes the title everywhere, not only in one playlist.

Sort, trim, list mode

Command	What it does
<code>ytpm sort PLAYLIST --by title</code>	Sparse sort. <code>--by</code> is title, date, duration, or channel.
<code>ytpm sort PLAYLIST --by title --dry-run</code>	Writes the list-mode file; 0 write quota .
<code>ytpm sort PLAYLIST --by title --reverse</code>	Reverse the chosen key (Z–A, newest first, longest first, ...).
<code>ytpm sort --resume</code>	Continues <code>sort_queue.json</code> after Pacific midnight.
<code>ytpm trim PLAYLIST [PLAYLIST...]</code>	Remove duplicate video ids.
<code>ytpm trim --all</code>	Trim every owned playlist.
<code>ytpm list export</code>	Export all playlists to list-mode text.
<code>ytpm list export PLAYLIST [PLAYLIST...]</code>	Export selected playlists only.
<code>ytpm list apply</code>	Apply <code>list_mode/*.txt</code> .
<code>ytpm list apply --dry-run</code>	Print planned deletes/inserts/moves; 0 writes .

Export and cleanup

Command	What it does
<code>ytpm export PLAYLIST</code>	JSON snapshot under the export folder (~ 2 reads).
<code>ytpm stats PLAYLIST</code>	Duration totals, longest/shortest, unique channels (~ 2 reads).
<code>ytpm cleanup PLAYLIST</code>	Find broken (placeholder) rows.
<code>ytpm cleanup PLAYLIST --remove</code>	Delete those playlist rows (50 × count). Optional <code>--private</code> / <code>--unlisted</code> to include those statuses in the scan.

Notes

- `--by` for sort: title, date, duration, channel. Add `--reverse` to invert.
- Dry-run sort/clear/apply still **reads** (list items). They do not spend **write** quota.
- Quota resets at midnight US Pacific. Google does not expose remaining Data API units; YTPM tracks a local ledger.
- `Run_YTPM_CLI.bat` uses `YTPM_VENV` from `.env` and installs missing libraries on startup (`ensure_deps.py`).
- Do not use unofficial YouTube web APIs to dodge quota.

See the button reference above (`docs/gui-buttons.md`) for GUI costs.

Limitations and YouTube API quota

Google does **not** sell extra Data API quota. The default allocation is 10,000 units per project per day. HTTP batching does not reduce unit cost. Extra Google Cloud projects used to multiply the 10k cap violate Google’s API terms. A team that shares one Desktop OAuth client (one Cloud project in the Windows zip) also shares that 10,000-unit pool.

You audit the project and ask for more units. Use the official paths below.

Form

YouTube API Services — Audit and Quota Extension

Docs

Quota and Compliance Audits

Console

YouTube Data API quotas

What YTPM will not do

- It cannot switch a playlist to Manual sort via the public API — that is Studio-only.
- It cannot update titles on videos you do not own. `Rename title...` uses `videos.update` and changes the title everywhere, not only in one playlist.
- It will not delete-and-rebuild a playlist to force order (that can empty the list if quota dies mid-run).
- Find Broken ignores playable unlisted videos; only YouTube placeholder titles count as broken.
- The sidebar quota meter is a local estimate. Google does not expose remaining Data API units.
- The video cache is in-memory only. Refresh drops it. It is not written to disk.
- Until a quota increase is approved, plan large sorts across days. Dry-run and List Mode save intended order at zero write cost.



Authored by **CN Sharad**

Isha Analytiks LLC · <https://www.ishaanalytiks.com> · info@ishaanalytiks.com

YouTube Playlist Manager (YTPM)

© Isha Analytiks LLC